Unidad 4

Funciones

Qué son las funciones y cómo se definen e invocan, el ámbito de las variables, los tipos de parámetros y de funciones, la recursividad y las callbacks de sort, forEach, map y filter.

Una **función** es un bloque de código con nombre que resuelve una tarea. Se define una vez y se usa tantas veces como haga falta.

```
function saludar(nombre) {
  return 'Hola, ' + nombre;
}

console.log(saludar('Ana'));    // → Hola, Ana
console.log(saludar('Luis'));   // → Hola, Luis
```

Ventajas

Sin funciones, una tarea que se repite en varios puntos del programa obliga a copiar su código en cada uno. Si después aparece un error, hay que encontrar y corregir todas las copias. Con una función, el código está en un solo sitio y cada punto la llama por su nombre.

Dividir un problema en funciones pequeñas que resuelven una parte cada una se llama **modularidad**. Aporta:

- **Reutilización**: una función probada sirve en otras partes del programa y en otros programas.
- **Legibilidad**: `calcularIVA(precio)` se entiende sin leer el cálculo.
- **Mantenimiento**: cada error se corrige en un solo sitio.

> ▶ **Ejemplo: un inicio de sesión dividido en funciones**

Anatomía de una función

```
function calcularCuadrado(numero) {
  const resultado = numero ** 2;
  return resultado;
}
```

Elemento	En el ejemplo	Para qué sirve
Palabra reservada	`function`	Inicia la definición
Identificador	`calcularCuadrado`	El nombre con el que se llama
Parámetros	`(numero)`	Los datos que recibe. Sin parámetros, `()`
Cuerpo	`{ … }`	Las instrucciones que ejecuta
Valor de retorno	`return resultado`	Lo que devuelve. Es opcional

Definición

Para usar una función, primero hay que definirla. La forma básica es la **declaración**: `function`, nombre, parámetros y cuerpo.

```
function avisoFormulario() {
  console.log('Cuidado, hay errores en el formulario');
}
```

Definirla no la ejecuta: este código no muestra nada.

> ▶ **Definición y declaración**

Invocación

Se invoca con su nombre y paréntesis, que son obligatorios aunque no reciba nada. Sin ellos se nombra la función, pero no se ejecuta.

```
avisoFormulario();
```

```
// → Cuidado, hay errores en el formulario
```

Los nombres de la definición son los **parámetros**; los valores que se pasan en la llamada, los **argumentos**:

```
function aviso(mensaje) {          // mensaje es el parámetro
  console.log(mensaje);
}

aviso('Error definido por el usuario');   // el texto es el argumento
// → Error definido por el usuario
```

Una llamada a una función que devuelve un valor se usa como ese valor: en una expresión, como argumento de otra función o de sí misma.

```
function raizCuadrada(numero) {
  return Math.sqrt(numero);
}

console.log(raizCuadrada(4));                    // → 2
console.log(raizCuadrada(4 + 5));                // → 3
console.log(raizCuadrada(4) + raizCuadrada(9));  // → 5
console.log(raizCuadrada(raizCuadrada(16)));     // → 2
console.log(raizCuadrada(11 - 3));               // → 2.8284271247461903
```

Una función puede llamar a otras, del lenguaje o propias:

```
function calcularMayor(lista) {
  let mayor = raizCuadrada(lista[0]);
  for (let i = 1; i < lista.length; i++) {
    if (raizCuadrada(lista[i]) > mayor) {
      mayor = raizCuadrada(lista[i]);
    }
  }
  return mayor;
}

console.log(calcularMayor([64, 128, 4, 1024, 16]));   // → 32
```

Valor de retorno

`return` devuelve un valor a quien llamó y **termina la función**: lo que haya detrás no se ejecuta. Sin `return`, la función devuelve `undefined`.

```
function saludar(nombre) {
  console.log('Hola, ' + nombre);
}

const resultado = saludar('Eva');   // → Hola, Eva
console.log(resultado);             // → undefined
```

Mostrar un valor con `console.log` no es devolverlo. Si quien llama necesita el resultado, la función tiene que hacer `return`.

Conviene saber qué tipo devuelve cada función. Si se usa donde se espera otro, JavaScript no da error: convierte y sigue.

```
function esPar(numero) {
  return numero % 2 === 0;
}

if (esPar(4)) {
  console.log('El número es par');   // bien: if espera un booleano
}

console.log(esPar(4) + 5);          // → 6   true se convierte en 1
```

Una función puede tener varios `return` y termina en el primero que se ejecute. En una función larga, un solo `return` al final facilita seguir el recorrido; en una corta, salir en cuanto se conoce la respuesta suele leerse mejor, como en `calificacion` de la unidad 2.

Ejemplo 1: aplicar un descuento

Escribe la función precioConDescuento(precio, porcentaje) que reciba un precio y un porcentaje de descuento y devuelva el precio nuevo. Por ejemplo, precioConDescuento(80, 25) devuelve 60. La función devuelve el número; no lo muestra por consola.

```
function precioConDescuento(precio, porcentaje) {

}

console.log(precioConDescuento(80, 25));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

Ejemplo 2: soluciones de una ecuación de segundo grado

Escribe la función ecuacionGrado2(a, b, c) que devuelva un array con las soluciones reales de la ecuación $ax^2 + bx + c = 0$, aplicando la fórmula $(-b \pm \sqrt{b^2 - 4ac}) / 2a$. Si el discriminante $b^2 - 4ac$ es positivo hay dos soluciones: primero la del signo + y después la del signo -. Si es 0 hay una sola, -b / 2a. Si es negativo no hay soluciones reales y el array queda vacío.

```
function ecuacionGrado2(a, b, c) {
  const soluciones = [];

  return soluciones;
}

console.log(ecuacionGrado2(1, -3, 2));
console.log(ecuacionGrado2(1, 2, 1));
console.log(ecuacionGrado2(1, 0, 1));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2

Ámbito y parámetros

Variables locales y globales

El **ámbito** de una variable es la parte del programa desde la que se ve. Una variable declarada dentro de una función es **local**: sólo existe dentro. Una declarada fuera de toda función es **global**: se ve desde cualquier parte.

```
const mensaje = 'Fuera de la función';

function mostrarAnuncio() {
  const mensaje = 'Dentro de la función';
  console.log(mensaje);
}

mostrarAnuncio();       // → Dentro de la función
console.log(mensaje);   // → Fuera de la función
```

Dentro de la función, la local tapa a la global; fuera, sólo existe la global. Si se quita la de dentro, la función ve la de fuera:

```
const mensaje = 'Fuera de la función';

function mostrarAnuncio() {
  console.log(mensaje);
}

mostrarAnuncio();       // → Fuera de la función
```

Y si se quita la de fuera, la local no se ve desde fuera:

```
function mostrarAnuncio() {
  const mensaje = 'Dentro de la función';
  console.log(mensaje);
}

mostrarAnuncio();       // → Dentro de la función
console.log(mensaje);   // ReferenceError: mensaje is not defined
```

Con `let` y `const` el ámbito es aún menor: el bloque `{ … }` donde se declaran, como se vio en la unidad 2.

Evita las variables globales: cualquier función puede cambiarlas, y cuando una acaba con un valor inesperado hay que revisar todo el

programa. Lo que una función necesita, mejor que lo reciba como parámetro y lo devuelva con `return`.

Paso por valor

Al llamar a una función, el valor de cada argumento **se copia** en su parámetro. Cambiar el parámetro no cambia la variable original:

```
const numero1 = 7;
const numero2 = 8;

function menor(primero, segundo) {
  primero = 10;
  segundo = 21;
  let elMenor = primero;
  if (segundo < primero) {
    elMenor = segundo;
  }
  return elMenor;
}

console.log(menor(numero1, numero2));   // → 10
console.log(numero1);                   // → 7
console.log(numero2);                   // → 8
```

Arrays, conjuntos y mapas como parámetro

Una variable no guarda el array, sino una **referencia** a él (unidad 3). Al pasarlo a una función se copia la referencia, así que el parámetro y la variable de fuera apuntan al **mismo** array, y los cambios se ven fuera:

```
const lista = [6, 2, 9, 5, 3];

function menorMarcado(numeros) {
  let elMenor = numeros[0];
  let posicion = 0;
  for (let i = 1; i < numeros.length; i++) {
    if (numeros[i] < elMenor) {
      elMenor = numeros[i];
      posicion = i;
    }
  }
```

```
  numeros[posicion] = -1;       // marca el menor
  return elMenor;
}

console.log(menorMarcado(lista));   // → 2
console.log(lista);                 // → [6, -1, 9, 5, 3]
```

Pasa igual con los conjuntos, los mapas y todos los objetos. A menudo se llama **paso por referencia**, aunque no lo es del todo:

> ▶ **Por qué no es un verdadero paso por referencia**

Si una función modifica lo que recibe, su nombre debe dejarlo claro: quien la llama puede no esperarlo.

Ejemplo 3: limpiar un mapa recibido como parámetro

Escribe la función limpiarCodigos(mapa) que reciba un Map cuyas claves son códigos postales (texto) y cuyos valores son localidades, y elimine de ese mismo mapa todas las entradas cuyo código no tenga exactamente cinco caracteres. La función no devuelve nada: modifica el mapa que recibe.

```
function limpiarCodigos(mapa) {

}

const localidades = new Map([
  ['41700', 'Dos Hermanas'],
  ['21440', 'Lepe'],
  ['1138', 'Tarifa'],
  ['41013', 'Sevilla'],
]);
limpiarCodigos(localidades);
console.log(localidades);
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 3

Parámetros por defecto

Un parámetro con `=` en la definición toma ese valor cuando la llamada no le da ninguno, o le da `undefined`:

```
function dividir(numerador, denominador = 1) {
  return numerador / denominador;
}

console.log(dividir(4));      // → 4     4 / 1
console.log(dividir(4, 2));   // → 2     4 / 2
console.log(dividir());       // → NaN   undefined / 1
```

Sin valor por defecto, un parámetro que no se pasa vale `undefined`.

Los argumentos se asignan **de izquierda a derecha**, así que los parámetros con valor por defecto van **al final**. Si van delante, no hay forma de saltárselos:

```
function dividir(numerador = 1, denominador) {
  return numerador / denominador;
}

console.log(dividir(4));      // → NaN   4 / undefined
```

Número variable de parámetros

Para recibir tantos argumentos como se pasen hay dos formas.

`arguments`. Dentro de una función declarada con `function`, `arguments` contiene todos los argumentos de la llamada:

```
function sumaTodo() {
  let suma = 0;
  for (let i = 0; i < arguments.length; i++) {
    suma += arguments[i];
  }
  return suma;
}

console.log(sumaTodo(11, 22, 33, 44, 55));   // → 165
```

Tiene índices y `length`, pero no es un array: le faltan `push`, `sort`, `includes` y los demás métodos. Y la cabecera `sumaTodo()` no dice que reciba nada.

Parámetro rest. Tres puntos delante del último parámetro recogen los argumentos que sobran en un array de verdad. Es la forma recomendada:

```
function sumaTodo(...numeros) {
  let suma = 0;
  for (const n of numeros) {
    suma += n;
  }
  return suma;
}

console.log(sumaTodo(11, 22, 33, 44, 55));   // → 165
console.log(sumaTodo());                     // → 0
```

Puede ir detrás de otros parámetros, pero siempre el último:

```
function presentar(titulo, ...nombres) {
  console.log(titulo + ': ' + nombres.length + ' personas');
}

presentar('Grupo A', 'Ana', 'Luis', 'Eva');   // → Grupo A: 3 personas
```

> ▶ **Rest y spread: los mismos tres puntos**

Ejemplo 4: adivinar letras con un parámetro rest

Escribe la función adivina(...letras) que reciba un número variable de letras y devuelva la palabra secreta con las letras acertadas en su sitio y un guion en cada posición no acertada. La palabra secreta es la constante SECRETA del código inicial. Por ejemplo, adivina('a', 'e', 'i', 'o', 'u', 'd', 'n') devuelve '-a--en-o'. Recoge las letras con un parámetro rest, no con arguments.

```
const SECRETA = 'sargento';

function adivina(...letras) {

}

console.log(adivina('a', 'e', 'i', 'o', 'u', 'd', 'n'));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 4

Tipos de funciones

En JavaScript una función es **un valor**, como un número o un array: se puede guardar en una variable, pasar como argumento o devolver con `return`. De ahí salen las distintas formas de escribirlas.

Declaración

La forma vista hasta ahora. Se puede llamar **antes** de la línea donde se define, porque JavaScript prepara las declaraciones antes de ejecutar nada (*hoisting*):

```
const resultado = multiplicar(7, 5);   // antes de definirla

function multiplicar(a, b) {
  return a * b;
}

console.log(resultado);                // → 35
```

Expresión

La función se guarda en una variable, y se llama por la variable:

```
const bienvenida = function sesionIniciada() {
  console.log('Bienvenido de nuevo');
};

bienvenida();       // → Bienvenido de nuevo
sesionIniciada();   // ReferenceError: sesionIniciada is not defined
```

El nombre `sesionIniciada` sólo existe dentro de la propia función. Al ser una asignación, termina en `;`.

Anónimas

Si el nombre no se usa desde fuera, se omite. Una función sin nombre es **anónima**:

```
const bienvenida = function () {
  console.log('Bienvenido de nuevo');
};

bienvenida();       // → Bienvenido de nuevo
```

A diferencia de una declaración, no existe hasta que se ejecuta la asignación. Usarla antes da error:

```
bienvenida();       // ReferenceError: Cannot access 'bienvenida'
                    // before initialization
const bienvenida = function () {
  console.log('Bienvenido de nuevo');
};
```

▶ new Function: existe, pero no se usa

Ejemplo 5: una función anónima en una variable

Crea una función anónima guardada en la constante invertida que reciba una cadena de texto y la devuelva invertida, de derecha a izquierda. Por ejemplo, invertida('almadraba') devuelve 'abardamla'. Defínela con const invertida = function (…) { … }, no con una declaración ni con una flecha.

```
const invertida = function (cadena) {

};

console.log(invertida('almadraba'));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 5

Callbacks

Una **callback** es una función que se pasa como argumento a otra para que la ejecute. Puede tener nombre o ser anónima.

```
const bienvenida = function () {
  return 'Bienvenido de nuevo, ';
};

const saludarUsuario = function (callback) {
  console.log(callback() + 'Javier');
};

saludarUsuario(bienvenida);   // → Bienvenido de nuevo, Javier
```

Se pasa sin paréntesis. `saludarUsuario(bienvenida())` pasaría el texto que devuelve, no la función, y al llamarlo daría `TypeError`.

Si es corta y se usa una vez, se escribe en la propia llamada:

```
saludarUsuario(function () {
  return 'Hola otra vez, ';
});
// → Hola otra vez, Javier
```

Si es larga, mejor definirla aparte con un nombre claro. Las callbacks se usan para ordenar y recorrer arrays (al final de este tema) y para responder a los clics del usuario.

Autoejecutables

Se ejecutan nada más definirse: la función va entre paréntesis y detrás, los paréntesis de la llamada, con sus argumentos si los tiene.

```
(function () {
  console.log('Bienvenido de nuevo');
})();
// → Bienvenido de nuevo

(function (usuario) {
  console.log('Bienvenido de nuevo, ' + usuario);
})('Javier');
// → Bienvenido de nuevo, Javier
```

Sirven para ejecutar algo una sola vez al arrancar sin dejar variables en el ámbito global:

```
(function () {
  const inicio = new Date();
  console.log('Inicio: ' + inicio.toLocaleString('es-ES'));
})();
// → Inicio: 19/10/2026, 10:42:07
```

Asignada a una variable, guarda **lo que devuelve**, no la función:

```
const saludo = (function (usuario) {
  return 'Bienvenido de nuevo, ' + usuario;
})('Javier');

console.log(saludo);   // → Bienvenido de nuevo, Javier
```

`saludo` es un texto: `saludo()` daría error.

Clausuras

Una función recuerda las variables que tenía alrededor al crearse, aunque la función que las contenía ya haya terminado. Esa función con sus variables es una **clausura** (*closure*).

```
function crearMensaje() {
  const texto = 'Hola de nuevo.';
  function mostrar() {
    console.log(texto);
  }
  return mostrar;
}

const mensaje = crearMensaje();   // crearMensaje ya ha terminado
mensaje();                        // → Hola de nuevo.
```

Su uso típico es guardar un estado privado, como un contador:

```
function crearContador() {
  let cuenta = 0;
```

```
  return function () {
    cuenta++;
    return cuenta;
  };
}

const contar = crearContador();
console.log(contar());   // → 1
console.log(contar());   // → 2

const otro = crearContador();
console.log(otro());     // → 1   cada contador tiene su propia cuenta
```

`cuenta` no es global, nadie más puede cambiarla, y aun así se conserva entre llamadas. Se aprovechará con los objetos y los eventos.

Flecha

Una forma corta de escribir funciones anónimas: sin `function` y con `=>` entre los parámetros y el cuerpo.

```
const saludar = function (nombre) {
  console.log('Hola, ' + nombre);
};

const saludarFlecha = (nombre) => {
  console.log('Hola, ' + nombre);
};
```

Abreviaturas:

- Si el cuerpo es **una sola expresión**, sobran las llaves y el `return`: el valor se devuelve solo.
- Con **un solo parámetro**, sobran sus paréntesis. Sin parámetros o con varios, son obligatorios.

```
const cuadrado = x => x * x;
const sumar = (a, b) => a + b;
const azar = () => Math.random();

console.log(cuadrado(5));      // → 25
console.log(sumar(2, 3));      // → 5
```

Con llaves, el `return` vuelve a ser obligatorio: `(a, b) => { a + b }` devuelve `undefined`.

Son la forma habitual de escribir callbacks cortas. No tienen `arguments`: para un número variable de argumentos se usa rest, `(...numeros) => numeros.length`.

Forma	Ejemplo	¿Antes de definirla?
Declaración	`function f(x) { … }`	sí
Expresión con nombre	`const f = function g(x) { … };`	no
Anónima en variable	`const f = function (x) { … };`	no
Flecha	`const f = (x) => …;`	no
Autoejecutable	`(function (x) { … })(5);`	se ejecuta al definirla

Ejemplo 6: una función flecha

Escribe una función flecha guardada en la constante masVocales que reciba dos cadenas y devuelva la que tiene más vocales. Cuenta como vocales a, e, i, o y u en minúscula. Si las dos tienen las mismas, devuelve la primera. Puedes escribir aparte otra función que cuente las vocales de una cadena.

```
const masVocales = (primera, segunda) => {

};

console.log(masVocales('casa', 'murcielago'));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 6

Recursividad

Una función es **recursiva** cuando se llama a sí misma. Sirve cuando el problema se define en términos de sí mismo: el factorial de 5 es 5 por el factorial de 4.

Caso base y caso recursivo

Toda función recursiva tiene:

- **Caso base**: la respuesta se conoce sin volver a llamar. Ahí se detiene.

- **Caso recursivo**: se llama con un problema **más pequeño**, más cerca del caso base.

El factorial, $n! = n \cdot (n - 1)!$, con caso base $1! = 1$:

```
function factorial(n) {
  if (n <= 1) {
    return 1;                    // caso base
  }
  return n * factorial(n - 1);   // caso recursivo
}

console.log(factorial(4));       // → 24
```

Cada llamada queda pendiente hasta que la siguiente le devuelve su resultado:

```
factorial(4)
4 * factorial(3)
4 * 3 * factorial(2)
4 * 3 * 2 * factorial(1)
4 * 3 * 2 * 1          ← caso base: ya no hay más llamadas
4 * 3 * 2
4 * 6
24
```

Sin caso base, o si no se acerca a él, la función se llama sin fin. Cada llamada pendiente ocupa sitio en la **pila de llamadas**, y al llenarse el programa se detiene:

```
function sinFinal(n) {
  return n * sinFinal(n - 1);    // no hay caso base
}

sinFinal(4);   // RangeError: Maximum call stack size exceeded
```

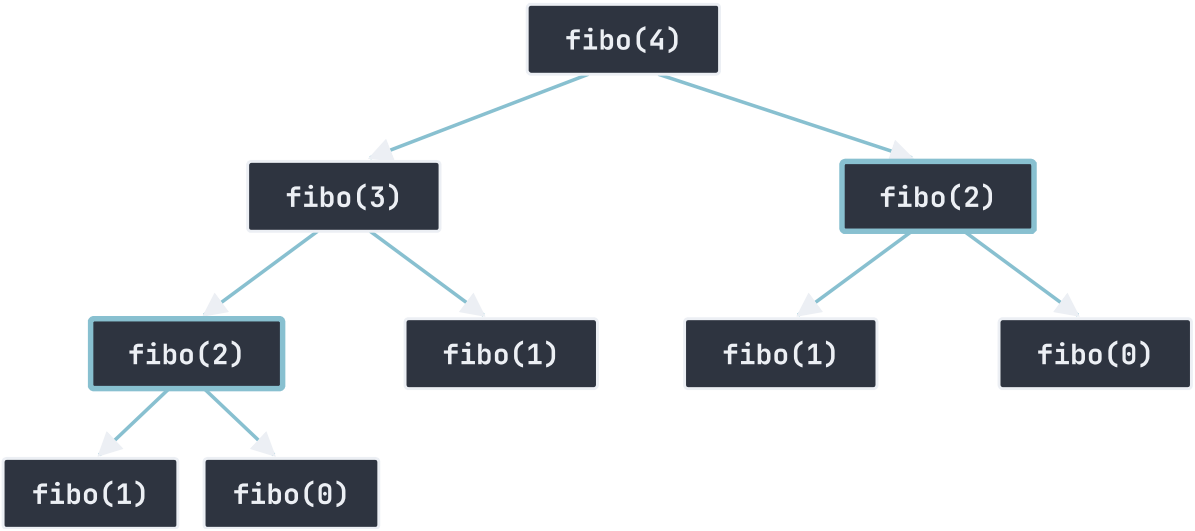
Fibonacci

Cada elemento es la suma de los dos anteriores: 1, 1, 2, 3, 5, 8, 13… Contando desde 0, las posiciones 0 y 1 son el caso base:

```
function fibo(n) {
  if (n < 2) {
    return 1;                          // posiciones 0 y 1
  }
  return fibo(n - 1) + fibo(n - 2);
}

for (let i = 0; i < 10; i++) {
  console.log(fibo(i));
}
// → 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 (uno por línea)
```

Cada llamada hace dos, así que forman un árbol. El diagrama muestra las llamadas de `fibo(4)`: cada flecha va de una llamada a las que hace.



Nueve llamadas para calcular fibo(4). Las de abajo del todo son casos base. fibo(2), con borde grueso, se calcula dos veces desde cero.

El resultado sube sumando: cada caso base devuelve 1, `fibo(2)` da 2, `fibo(3)` da 3 y `fibo(4)` da 5.

Coste

Las llamadas repetidas se multiplican: `fibo(30)` hace casi 2,7 millones, y un bucle lo resuelve en 30 pasos:

```
function fiboIterativo(n) {
  let anterior = 1;
  let actual = 1;
```

```
  for (let i = 2; i <= n; i++) {
    const siguiente = anterior + actual;
    anterior = actual;
    actual = siguiente;
  }
  return actual;
}
```

Todo problema recursivo se puede resolver también con bucles, que suelen gastar menos tiempo y memoria. Usa la recursividad cuando deja la solución mucho más clara, como al recorrer estructuras que contienen otras de su mismo tipo (carpetas dentro de carpetas), y su coste es aceptable.

Ejemplo 7: ¿es palíndroma?

Escribe una función recursiva palindroma(cadena) que devuelva true si la cadena se lee igual de izquierda a derecha que de derecha a izquierda, como 'SOS', 'rajar' u 'orejero', y false si no. Distingue mayúsculas de minúsculas. Resuélvelo con recursividad: sin bucles y sin invertir la cadena.

```
function palindroma(cadena) {

}

console.log(palindroma('orejero'));
console.log(palindroma('orejas'));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

> ▶ Ver solución del ejemplo 7

Funciones con arrays, conjuntos y mapas

Algunos métodos de la unidad 3 reciben una callback que llaman por cada elemento. Con funciones flecha, cada uno sustituye a un bucle.

Ordenar con un comparador

`sort()` sin argumentos ordena como texto, por código Unicode: las mayúsculas van antes que las minúsculas, la «ñ» después de la «z» y los números cifra a cifra.

```
const palabras = ['Casado', 'casa', 'prueba', 'zancos', 'ñam'];
palabras.sort();
// → ['Casado', 'casa', 'prueba', 'zancos', 'ñam']

const numeros = [10, 9, 1, 100];
numeros.sort();
// → [1, 10, 100, 9]          '100' va antes que '9'
```

Para otro criterio, `sort` recibe un **comparador**: una función con dos elementos, `a` y `b`, que devuelve un número:

Devuelve	Resultado
negativo	`a` va antes que `b`
positivo	`b` va antes que `a`
0	se quedan como estaban

Con números, basta restar. `sort` sólo mira el **signo** de lo que devuelve el comparador, y `a - b` da justo el signo que pide la tabla:

`a` y `b`	`a - b`	Resultado
`a` menor que `b`	negativo: `1 - 9` da `-8`	`a` va antes
`a` mayor que `b`	positivo: `10 - 9` da `1`	`b` va antes

a y b	a - b	Resultado
iguales	0	se quedan como estaban

El menor queda siempre delante: orden ascendente. El tamaño del número da igual, `-8` y `-1` dicen lo mismo, «a va antes». Restando al revés, `b - a`, se invierten todos los signos y el orden pasa a ser descendente.

```
const numeros = [10, 9, 1, 100];

numeros.sort((a, b) => a - b);    // → [1, 9, 10, 100]   ascendente
numeros.sort((a, b) => b - a);    // → [100, 10, 9, 1]   descendente
```

`sort` no compara todos con todos ni recorre el array en un orden fijo: decide él qué parejas pregunta y coloca los elementos según las respuestas. Para verlo, basta un `console.log` dentro del comparador:

```
const numeros = [10, 9, 1, 100];
numeros.sort((a, b) => {
  console.log(`a = ${a}, b = ${b}, a - b = ${a - b}`);
  return a - b;
});
```

En Node.js y en Chrome pregunta cinco veces:

Paso	a	b	a - b	Conclusión	Orden conocido
1	9	10	-1	9 va antes que 10	[9, 10]
2	1	9	-8	1 va antes que 9	[1, 9, 10]
3	100	1	99	100 va después de 1	[1, 9, 10], el 100 tras el 1
4	100	9	91	100 va después de 9	[1, 9, 10], el 100 tras el 9

Paso	a	b	a - b	Conclusión	Orden conocido
5	100	10	90	100 va después de 10	[1, 9, 10, 100]

En los pasos 1 y 2, el 9 y el 1 son menores que el anterior y van delante. Del 3 al 5, `sort` busca el sitio del 100: va detrás del 1, del 9 y del 10, así que va al final. `a` no es siempre el primero del array: en el paso 1 es el 9, que está en la segunda posición. Otro navegador puede preguntar otras parejas o en otro orden, pero el resultado es el mismo, porque el comparador siempre responde igual a la misma pareja.

Por longitud:

```
const palabras = ['Casado', 'casa', 'prueba', 'zancos', 'ñam'];
palabras.sort((a, b) => a.length - b.length);
// → ['ñam', 'casa', 'Casado', 'prueba', 'zancos']
```

En orden alfabético español, con la «ñ» en su sitio y sin separar mayúsculas, con `localeCompare`:

```
const palabras = ['Casado', 'casa', 'prueba', 'zancos', 'ñam'];
palabras.sort((a, b) => a.localeCompare(b, 'es'));
// → ['casa', 'Casado', 'ñam', 'prueba', 'zancos']
```

Dos `sort` seguidos no combinan criterios: el segundo deshace el primero. Para desempatar, un solo comparador con `||`, que pasa al segundo criterio cuando el primero da 0:

```
const palabras = ['pera', 'kiwi', 'uva', 'higo', 'lima'];
palabras.sort((a, b) =>
  a.length - b.length || a.localeCompare(b, 'es'));
// → ['uva', 'higo', 'kiwi', 'lima', 'pera']
```

Ejemplo 8: pares primero, pero en orden

Escribe la función paresPrimero(lista) que ordene un array de enteros positivos con sort y un comparador, de forma que primero vayan los pares de menor a mayor y después los impares, también de menor a mayor. Como sort modifica el array, la función ordena el array recibido y lo devuelve.

```
function paresPrimero(lista) {

}

console.log(paresPrimero([5, 7, 3, 1, 4, 9, 2, 6, 8]));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 8

forEach

Llama a la callback por cada elemento. En los arrays recibe el elemento y, si se pide, su posición:

```
const lista = [12, 334, 111];

lista.forEach((elemento, posicion) => {
  console.log(`Posición ${posicion}: ${elemento}`);
});
// → Posición 0: 12
```

```
// → Posición 1: 334
// → Posición 2: 111
```

En los conjuntos, cada elemento:

```
const conjunto = new Set([12, 334, 111]);

conjunto.forEach((elemento) => {
  console.log(`Elemento: ${elemento}`);
});
// → Elemento: 12
// → Elemento: 334
// → Elemento: 111
```

En los mapas, el **valor primero y la clave después**, al revés que en `set(clave, valor)`:

```
const mapa = new Map([['a', 12], ['b', 334], ['c', 111]]);

mapa.forEach((valor, clave) => {
  console.log(`Clave: ${clave} / Valor: ${valor}`);
});
// → Clave: a / Valor: 12
// → Clave: b / Valor: 334
// → Clave: c / Valor: 111
```

Se salta los huecos, como `for … in`. No se puede cortar: `break` no vale dentro de la callback y `return` sólo pasa al siguiente elemento. Para salir antes, `for … of`.

map

Devuelve un array **nuevo** del mismo tamaño, con lo que devuelve la callback para cada elemento. El original no cambia:

```
const sinIVA = [12.45, 34.42, 99.9, 49.95];
const conIVA = sinIVA.map((precio) => (precio * 1.21).toFixed(2));

console.log(sinIVA);   // → [12.45, 34.42, 99.9, 49.95]
console.log(conIVA);   // → ['15.06', '41.65', '120.88', '60.44']
```

`toFixed(2)` devuelve **texto**. Para seguir calculando, conviértelo con `Number`. El método `map` no tiene nada que ver con la estructura `Map`.

filter

Devuelve un array **nuevo** con los elementos para los que la callback devuelve `true`. El original no cambia:

```
const playas = ['Riazor', 'Orzán', 'Matadero', 'Oza', 'Bastiagueiro'];
const filtradas = playas.filter((playa) => playa.length !== 6);

console.log(filtradas);   // → ['Orzán', 'Matadero', 'Oza', 'Bastiagueiro']
```

`map` y `filter` devuelven arrays, así que se encadenan:

```
const notas = [4, 7.5, 9, 3, 6];
const aprobadas = notas
  .filter((nota) => nota >= 5)
  .map((nota) => nota * 10);
// → [75, 90, 60]
```

Método	Devuelve	¿Modifica el original?
`sort(comparador)`	el mismo array, ordenado	sí
`forEach(callback)`	`undefined`	no, salvo que lo haga la callback
`map(callback)`	un array nuevo del mismo tamaño	no
`filter(callback)`	un array nuevo con los que cumplen	no

Ejemplo 9: quitar las palabras con tilde

Escribe la función sinTildes(palabras) que, usando filter, devuelva un array nuevo con las palabras del array recibido que no contienen ninguna vocal con tilde (á, é, í, ó, ú, en minúscula o en mayúscula). El array original no debe cambiar.

```
function sinTildes(palabras) {

}

console.log(sinTildes(['árbol', 'casa', 'canción', 'mesa']));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 9

De este módulo

T4.1 Proyecto: gestor de entrenamientos
Un programa de consola en varios archivos que crea planes de entrenamiento y los gestiona desde un menú, con funciones que no mutan los datos, recursividad y funciones de orden superior.

TO4.1 Ejercicios: tipos de funciones
Ocho programas, cada uno con un tipo de función distinto, de la declaración con hoisting a la recursividad y los métodos sort, map,

filter y forEach.
